Contents

Exercise 1: Introduction to Numpy…… 2

Exercise 2: Matrix Operation ……..…….. 10

Exercise 3: Programs using Matplotlib …….. 14

Exercise 4: Introduction to Pandas…… 26

Exercise 5: Implementation of KNN…….. 32

Exercise 6: Implementation of Naive Bayes Classifier……. 36

Exercise 7: Implementation of Decision Tree ……..… 38

Exercise 8: Implementation of K-Means Clustering……. 42

Exercise 9: Linear Regression…… 46

Exercise 10: Multiple Regression…… 50

Date:

***Exercise 1*: Introduction to Numpy**

Aim:

1. Write a NumPy program to convert Fahrenheit degree to centigrade degree. Fahrenheit values are stored in a NumPy array.
2. Write a NumPy program to create a random array with 1000 elements and compute the average, variance, standard deviation of the array elements.
3. Write a NumPy program to convert angles from degrees to radians for all elements in a given array.
4. Write a NumPy program to calculate the difference between neighbouring elements, element-wise of a given array.
5. Write a NumPy program to sort a given complex array using the real part first, then the imaginary part.
6. Write a NumPy program to create a 1D array with random numbers and print the values greater than 4.
7. Write a NumPy program to create a 1D array with random numbers and search a particular number given by the user.
8. Write a NumPy program to save a given array to a text file and load it.
9. Write a NumPy program to create an array of all the even integers from 30 to 60.
10. Write a NumPy program to compute sum of all elements ,sum of each column and sum of each row of a given array.

Program 1:

```
import numpy as np
fht = np.array([100,98.4,91.50,99.78,101.5])
degree=np.array([])
for temp in fht:
    degreeval=(temp-32)*(5/9)
    degree = np.append(degree,degreeval)
print("Fahrenheit values",fht)
print("Centigrade values",degree)
```

Program 2:

```
arr =  np.array([23,55,29,56,28,86,12])
mean = np.mean(arr)
variance = np.var(arr)
std = np.std(arr)
print("Array=",arr)
print("Mean=",mean," Variance=",variance," Standard deviation= ",std)
```

Output

Output 1:

```
Fahrenheit values [100.    98.4   91.5   99.78 101.5 ]
Centigrade values [37.77777778 36.88888889 33.05555556 37.65555556 38.61111111]
```

Output 2:

```
Array= [23 55 29 56 28 86 12]
Mean= 41.285714285714285  Variance= 560.4897959183673  Standard deviation=
23.674665698133253
```

Output 3:

```
Degrees =  [ 32  40 180 360  45  93]
Radian values = [0.55850536 0.6981317  3.14159265 6.28318531 0.78539816 1.6231562 ]
```

Output 4:

```
Array [12 34  6 28 46 90  3]
Difference [ 22 -28  22  18  44 -87]
```

Output 5:

```
Array before sorting= [ 7.+5.j  4.+3.j 12.+2.j  1.+8.j  1.+9.j]
After sorting= [ 1.+8.j  1.+9.j  4.+3.j  7.+5.j 12.+2.j]
```

Output 6:

```
Array= [-12 -31 -18   1  29  15 -13  38 -32  29]
Elements greater than 4
[29 15 38 29]
```

Output 7:

```
Enter a number
12
[-10 -10  -2  29  29  39   8  30  26   2]
Number is not present in the array


Enter a number
[-10 -10  -2  29  29  39   8  30  26   2]
Number is present in the array
```

Output 8:

```
Writing the array to the file
File content is
array([   4.,    5.,    4.,    6.,   11., 2235.,   56.])
```

Program 3:

```
degrees = np.array([32,40,180,360,45,93])
radians  = np.deg2rad(degrees)
print("Degrees = ",degrees)
print("Radian values =",radians)
```

Program 4:

```
arr = np.array([12,34,6,28,46,90,3])
diffs =  np.diff(arr)
print("Array",arr)
print("Difference",diffs)
```

Program 5:

```
arr =np.array([complex(7,5),complex(4,3),complex(12,2),complex(1,8),complex(1,9)])
sorted = np.sort(arr)
print("Array before sorting=",arr)
print("After sorting=",sorted)
```

Program 6:

```
arr=np.random.randint(low=-40,high=40,size=10)
print("Array=",arr)
res =np.where(arr > 4)
print("Elements greater than 4")
for ele in res:
    print(arr[res])
```

Program 7:

```
arr=np.random.randint(low=-10, high=40, size=10)
print(arr)
num=int(input("Enter a number"))
res=np.where(arr==num)
if(len(res[0])==0):
    print("Number is not present in the array")
else:
    print("Number is present in the array")
```

Program 8:

```
print("Writing the array to the file")
np.savetxt(fname="test.txt",X=arr)
print("File content is")
np.loadtxt(fname="test.txt")
```

Program 9:

```
arr =  np.arange(30,61,2)
print("Array of all the even integers from 30 to 60.")
print(arr)
```

Output 9:

```
Array of all the even integers from 30 to 60.
[30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60]
```

Output 10:

```
Array= [[1 2 3]
 [4 5 6]
 [7 8 9]]
Sum of elements= 45
Column wise sum= [12 15 18]
Row wise sum= [ 6 15 24]
```

Program 10:

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
print("Array=",arr)
# Sum of all elements
total_sum = np.sum(arr)
print("Sum of elements=",total_sum)
# Sum of each column (axis=0)
col_sum = np.sum(arr, axis=0)
print("Column wise sum=",col_sum)
# Sum of each row (axis=1)
row_sum = np.sum(arr, axis=1)
print("Row wise sum=",row_sum)
```

Result:

The program has been successfully executed and the output is verified

Output

Output 1:

```
Enter the order of the matrix
2
Matrix 1

Enter element 0,0
1
Enter element 0,1
2
Enter element 1,0
3
Enter element 1,1
4

Matrix 2

Enter element 0,0
2
Enter element 0,1
3
Enter element 1,0
4
Enter element 1,1
5

Matrices are
[[1 2]
[3 4]]
[[2 3]
[4 5]]
Dot Product= [[10 13] [22 29]]
```

Output 2:

```
Transpose of matrix 1 = [[1 3] [2 4]]
```

Output 3:

```
Trace of matrix 1 = 5
```

Output 4:

```
Rank of matrix 1 = 2
```

Output 5:

```
Determinant of matrix 1= -2.0000000000000004
```

***Exercise 2*: Matrix Operations**

Aim:

Write a NumPy program to create two matrices (read values from user) and find the following

1. Dot Product
2. Transpose
3. Trace
4. Rank
5. Determinant
6. Inverse
7. Eigen values and eigen vectors

Program 1:

```
import numpy as np
from numpy import linalg
def matrixread(n):
    matrix = []
    for i in range(0,n):
        temp=[]
        for j in range(0,n):
            num=int(input("Enter element "+str(i)+","+str(j)))
            temp.append(num)
        matrix.append(temp)
    return matrix

n=int(input("Enter the order of the matrix"))
print("Matrix 1")
matrix1=np.array(matrixread(n))
print("Matrix 2")
matrix2=np.array(matrixread(n))
print("Matrices are")
print(matrix1)
print(matrix2)
```

Program 2:

```
print("Transpose of matrix 1 =", np.transpose(matrix1))
```

Program 3:

```
print("Trace of matrix 1 =", np.trace(matrix1))
```

Program 4:

```
print("Rank of matrix 1 =",linalg.matrix_rank(matrix1))
```

Output 6:

```
Inverse of matrix 1= [[-2.   1. ]
[ 1.5 -0.5]]
```

Output 7:

```
Eigen values and Eigen vectors of matrix 1
EigResult(eigenvalues=array([-0.37228132,  5.37228132]),
eigenvectors=array([[-0.82456484, -0.41597356],
       [ 0.56576746, -0.90937671]]))
```

Program 5:

```
print("Determinant of matrix 1=", np.linalg.det(matrix1))
```

Program 6:

```
print("Inverse of matrix 1=", linalg.inv(matrix1))
```

Program 7:

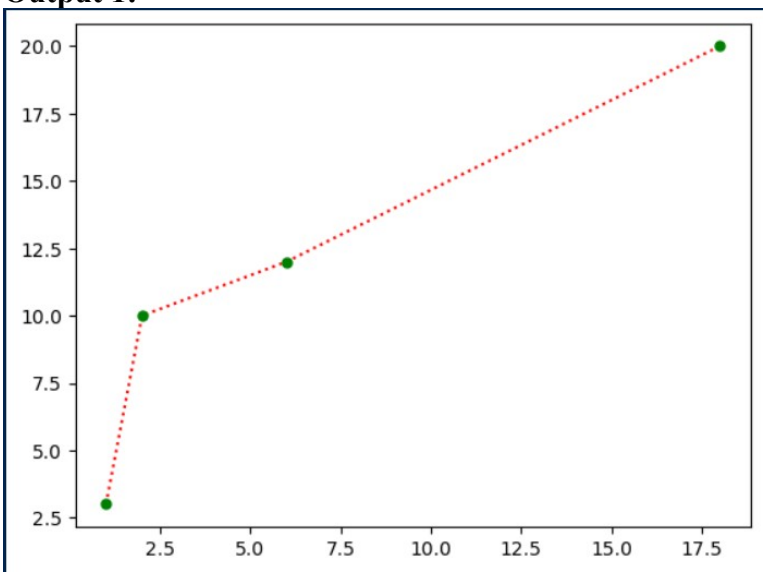
```
print("Eigen values and Eigen vectors of matrix 1", linalg.eig(matrix1))
```

Result:

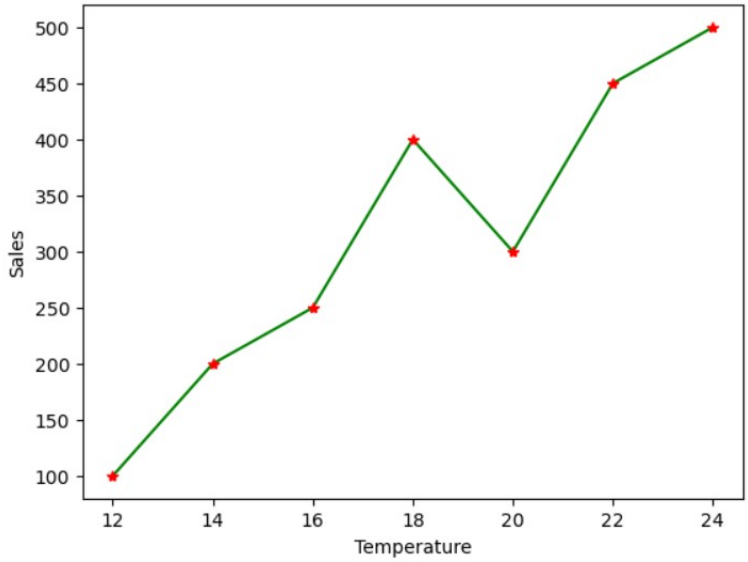
The program has been successfully executed and the output is verified.

Output

Output 1:



Output 2:



Exercise 3: Programs using Matplotlib

Aim:

1. Draw a line from position (1,3) and (2,10) then to (6,12) and finally to position (18,20).
   Mark each point with a beautiful green color and set line colour to red and line style dotted.
2. Draw a plot for the following data

Temperature in degree	Sales
12	100
14	200
16	250
18	400
20	300
22	450
24	500

3. Write a python program to draw a line using given axis values taken from a text file, with suitable labels in x and y axis and a title.
4. Write a python program to plot two or more lines on same plot with suitable legends for each line.
5. Write a python program to create multiple plots.
6. Consider the following data.

Programming language	Popularity
Java	22.2
Python	17.6
PHP	8.8
JavaScript	8
C#	77
C++	6.7

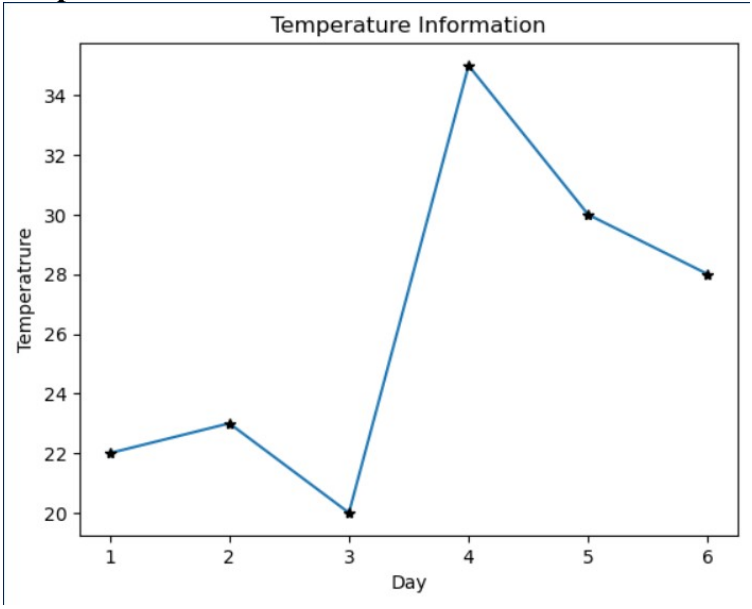
   i. Write a python program to display a bar chart of the popularity of programming languages.
   ii. Write a python program to display a horizontal bar chart of the popularity of programming languages. (Make it red color).
   iii. Write a python program to display a bar chart of the popularity of programming languages. Use different colors for each bar.

7. Write a python program to create bar plots of scores by group and gender. Use multiple x values on the same chart for men and women.
8. Write a python program to create a pie chart on the popularity of programming languages.
9. Write a python program to create a pie chart of gold medal achievements of five most successful countries in 2016 Summer Olympics. Read the data from a csv file.
10. Write a python Program to draw a scatter plot comparing two subject marks of Mathematics and Science. Use marks of 10 students.

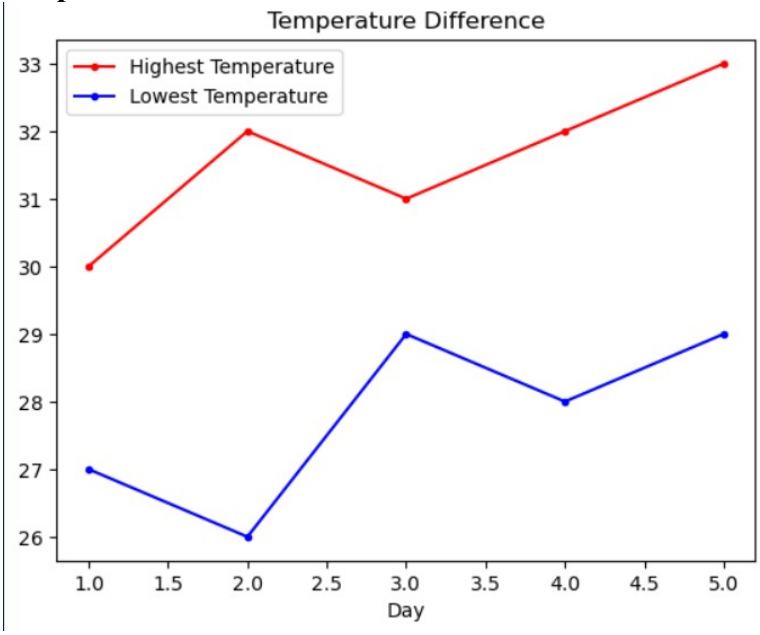
Program 1:

```
import matplotlib.pyplot as plt
import numpy as np
x_data = np.array([1,2,6,18])
y_data = np.array([3,10,12,20])
plt.plot(x_data,y_data,ls=":",marker=".",ms=10,mfc="green",mec="green",color="red")
```

Output 3:



Output 4:



Program 2:

```
x_data = np.array([12,14,16,18,20,22,24])
y_data = np.array([100,200,250,400,300,450,500])
plt.xlabel("Temperature")
plt.ylabel("Sales")
plt.plot(x_data,y_data,ls="-",color="green",marker="*",mfc="red",mec="red")
```

Program 3:

```
file=open("exercise3.3.txt")
x_data =np.array(list(map(int,file.readline().strip().split(","))))
y_data =np.array(list(map(int,file.readline().strip().split(","))))
file.close()
plt.title("Temperature Information")
plt.xlabel("Day")
plt.ylabel("Temperatrure")
plt.plot(x_data,y_data,ls="-",marker="*",mec="black",mfc="black")
```

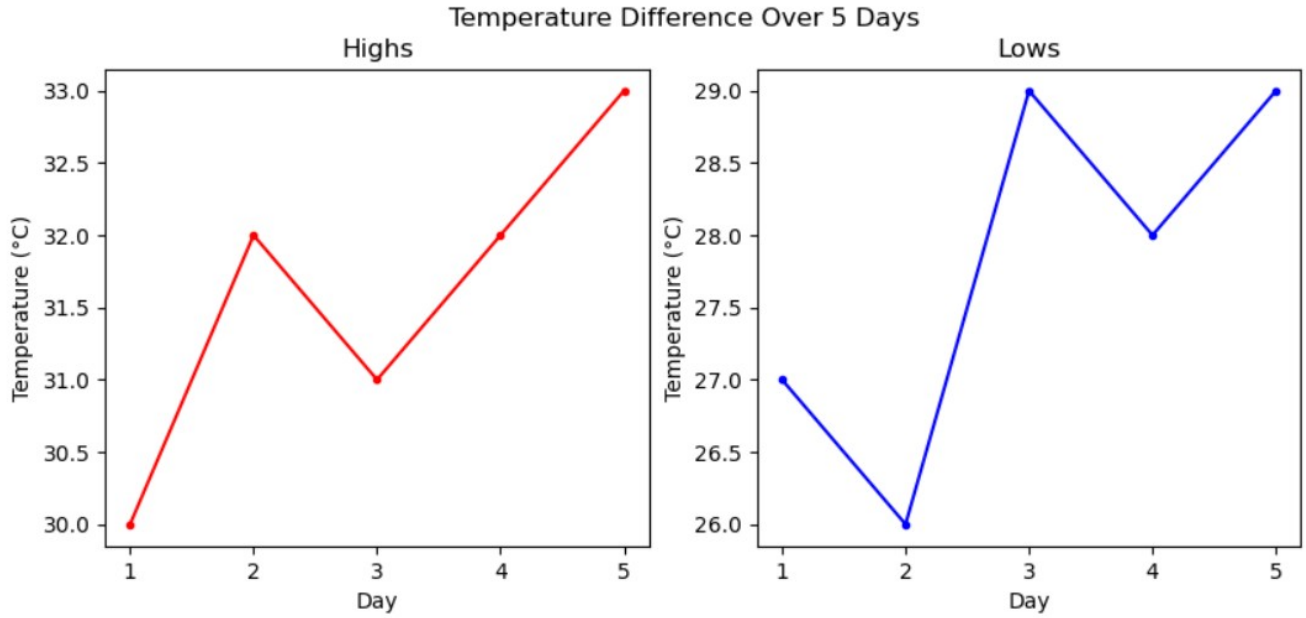
Program 4:

```
x = np.array([1,2,3,4,5])
y_1= np.array([30,32,31,32,33])
y_2=np.array([27,26,29,28,29])
plt.xlabel("Day")
plt.plot(x,y_1,color="red",marker=".",mfc="red",label="Highest Temperature")
plt.plot(x,y_2,color="blue",marker=".",mfc="blue",label="Lowest Temperature")
plt.title("Temperature Difference")
plt.legend() # adding legend to display labels
plt.show()
```

Program 5:

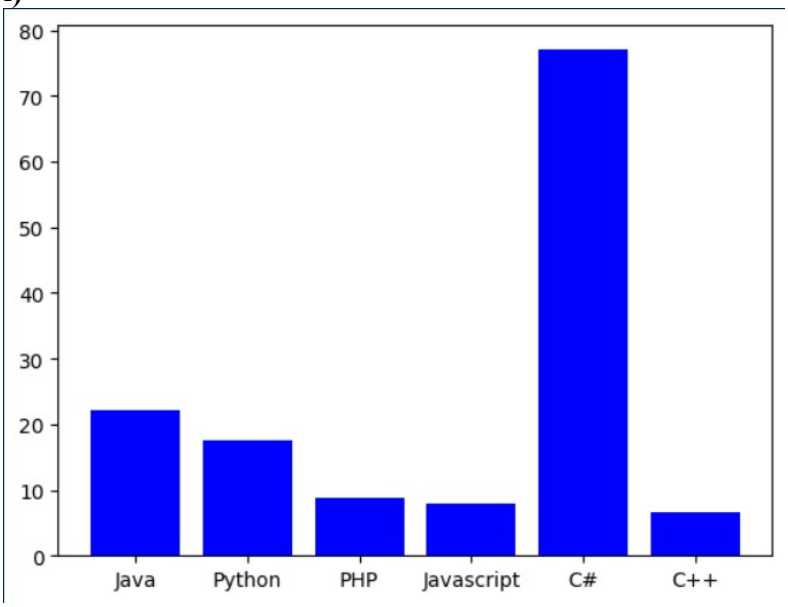
```
x = np.array([1, 2, 3, 4, 5])
y_1 = np.array([30, 32, 31, 32, 33])
y_2 = np.array([27, 26, 29, 28, 29])
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(x, y_1, color="red", marker=".", mfc="red", label="Highest Temperature")
plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.title("Highs")
plt.subplot(1, 2, 2)
plt.plot(x, y_2, color="blue", marker=".", mfc="blue", label="Lowest Temperature")
plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.title("Lows")
plt.suptitle("Temperature Difference Over 5 Days")
plt.show()
```

Output 5:



Output 6:

i)



Program 6:

i)

```
x_data = ["Java","Python","PHP","Javascript","C#","C++"]
y_data=[22.2,17.6,8.8,8,77,6.7]
plt.bar(x_data,y_data,color="blue")
```

ii)

```
plt.barh(x_data,y_data,color="red")
plt.title("Programming Languages Popularity")
```

iii)

```
colors = ['red', 'green', 'blue', 'orange', 'purple']
plt.bar(x_data,y_data,color=colors)
```

Program 7:

```
x_data = np.arange(1,6) *3
y1= np.array([22,30,35,35,26])
y2= np.array([25,22,30,35,29])
bar_width=0.4
plt.bar(x_data-bar_width,y1,label="Men",color='Blue')
plt.bar(x_data+bar_width,y2,label="Women",color='Pink')
plt.legend()
plt.title("Scores by person")
```

Program 8:

```
x_data = ["Java","Python","PHP","Javascript","C#","C++"]
y_data=[30,40,10,10,5,5]
plt.pie(y_data,labels=x_data)
```

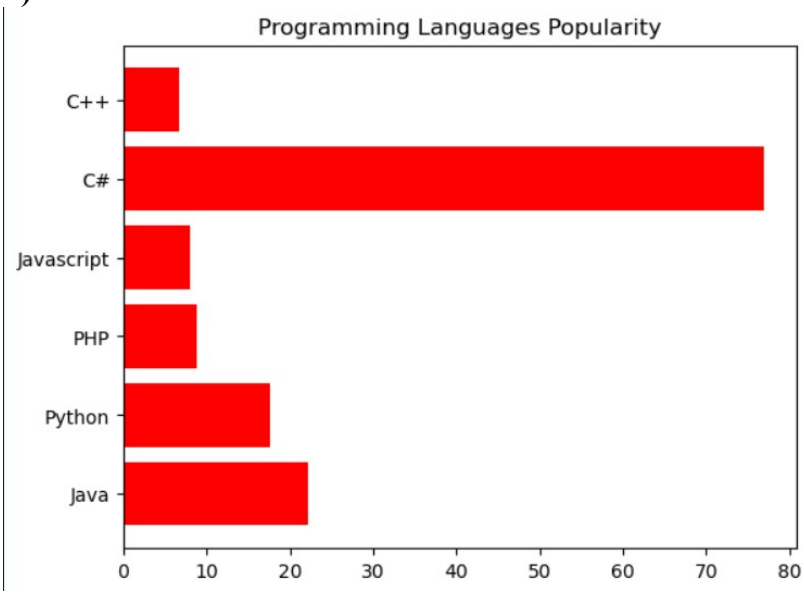
Program 9:

```
import pandas as pd
df=pd.read_csv("medal.csv",delimiter=":")
countries = df['Country'].to_numpy()
medals=df['Gold_Medal'].to_numpy()
plt.pie(medals,labels=countries,autopct='%1.1f%%')
```

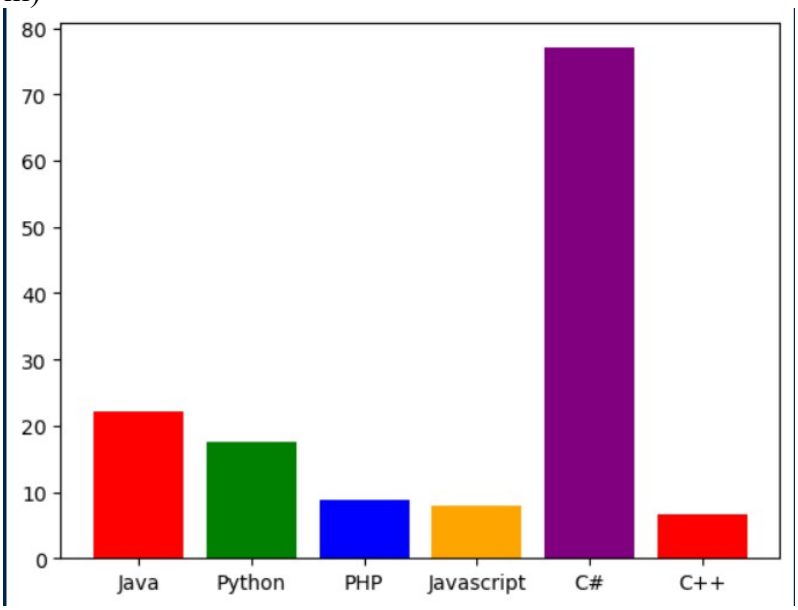
Program 10:

```
math_marks = np.array([88,92,80,89,100,80,60,100,80,34])
science_marks = np.array([35,79,79,48,100,88,32,45,20,30])
marks_range = np.array([10,20,30,40,50,60,70,80,90,100])
plt.xlabel("Marks Range")
plt.title("Math marks vs Scince marks")
plt.scatter(marks_range,math_marks,color="red",label="Maths")
plt.scatter(marks_range,science_marks,color="green",label="Science")
plt.legend()
plt.show()
```

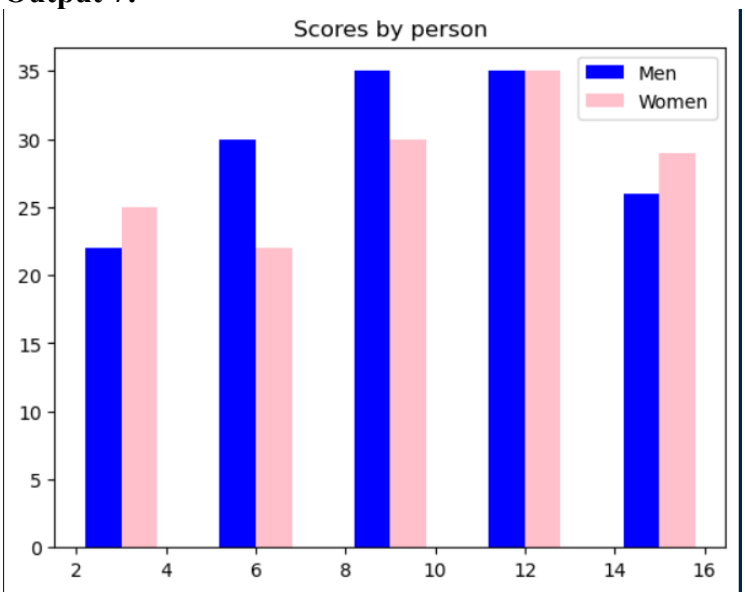
ii)


Programming Languages Popularity
C++
C#
Javascript
PHP
Python
Java
0
10
20
30
40
50
60
70
80

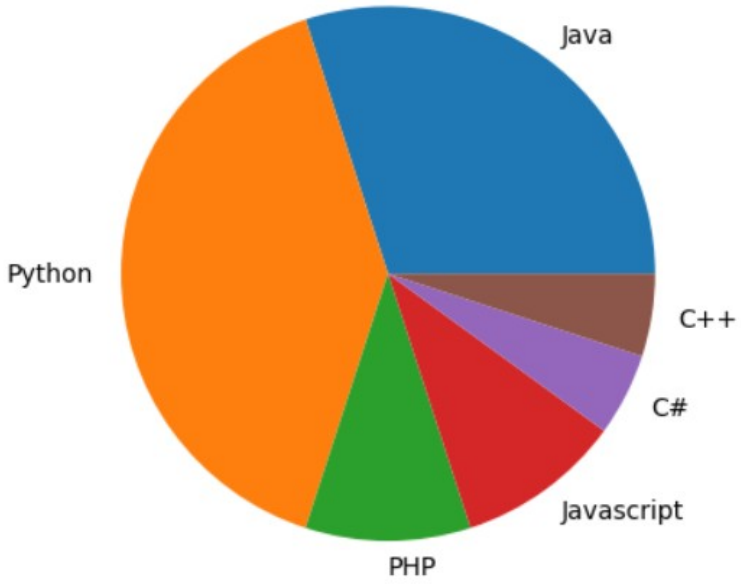
iii)


80
70
60
50
40
30
20
10
0
Java
Python
PHP
Javascript
C#
C++

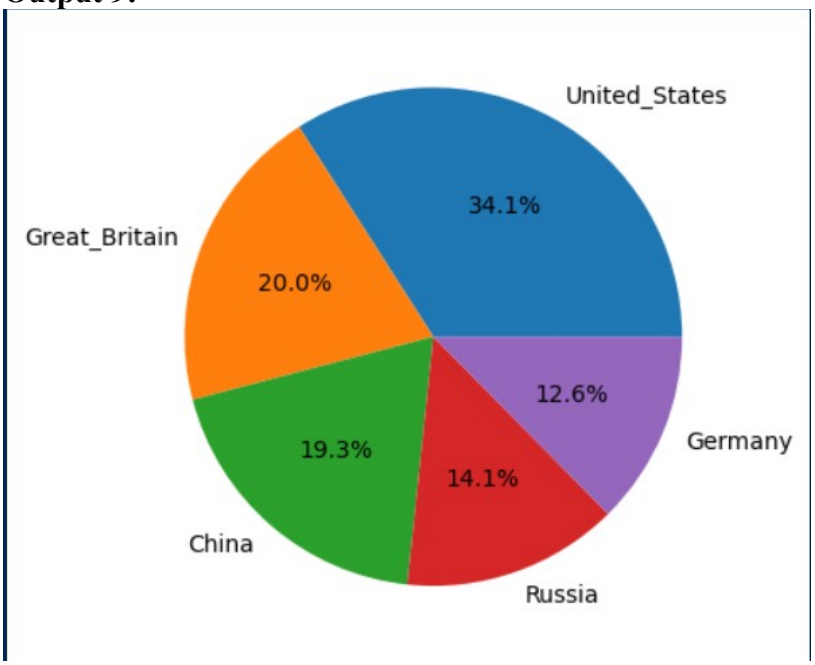
Output 7:



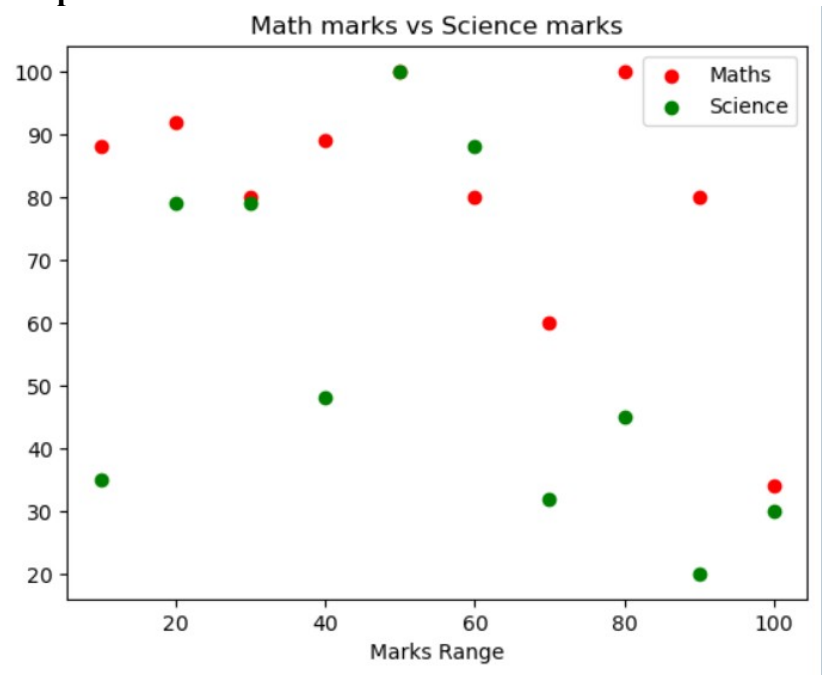
Output 8:



Output 9:



Output 10:



Result:

The program has been successfully executed and the output is verified.

Output

Output 1:

```
0    1
1    4
2    6
3    3
dtype: int64
```

Output 2:

```
Date Series:
 DatetimeIndex(['2021-05-01', '2021-05-02', '2021-05-03', '2021-05-04',
                '2021-05-05', '2021-05-06', '2021-05-07', '2021-05-08',
                '2021-05-09', '2021-05-10', '2021-05-11', '2021-05-12'],
               dtype='datetime64[ns]', freq='D')
```

Output 3:

```
Dictionary to DataFrame:
       Name  Age       City
0    Alice   24      Delhi
1      Bob   30     Mumbai
2  Charlie   28  Bangalore
```

Output 4:

```
   ID   Name  Score
0   1  Alice     10
1   2    Bob     15
2   3    Don     25
```

Output 5:

```
         Country  Gold_Medal
0  United_States          46
1  Great_Britain          27
2          China          26
3         Russia          19
4        Germany          17
```

Output 6:

```
Data frame
   ID   Name  Score
0   1  Alice     15
1   2    Bob     15
2   3    Don     45
Soretd Dataframe
   ID   Name  Score
2   3    Don     45
0   1  Alice     15
1   2    Bob     1
```

***Exercise 4*: Introduction to Pandas**

Aim:

1. Write a python program to implement List-to-Series Conversion.
2. Write a python program to Generate the series of dates from 1st May, 2021 to 12th May, 2021 (both inclusive).
3. Given a dictionary, convert it into corresponding dataframe and display it.
4. Given a 2D List, convert it into corresponding dataframe and display it.
5. Given a CSV file, read it into a dataframe and display it.
6. Given a dataframe, sort it by multiple columns.
7. Given a dataframe with custom indexing, convert and it to default indexing and display it.
8. Given a dataframe, select first 2 rows and output them.
9. Given is a dataframe showing name, occupation, salary of people. Find the average salary per occupation.
10. Given a dataframe with NaN Values, fill the NaN values with 0.
11. Given is a dataframe showing Company Names (cname) and corresponding Profits (profit). Convert the values of Profit column such that values in it greater than 0 are set to True and the rest are set to False.
12. Given are 2 dataframes, with one dataframe containing Employee ID (eid), Employee Name (ename) and Stipend (stipend) and the other dataframe containing Employee ID (eid) and designation of the employee (designation). Output the Dataframe containing Employee ID (eid), Employee Name (ename), Stipend (stipend) and Position (position).

Program 1:

```
listdata =  [1,4,6,3]
series = pd.Series(listdata)
print(series)
```

Program 2:

```
print(pd.date_range("2021-05-01","2021-05-12"))
```

Program 3:

```
data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [24, 30, 28],
    "City": ["Delhi", "Mumbai", "Bangalore"]
}
df = pd.DataFrame(data)
print("Dictionary to DataFrame:\n", df)
```

Output 7:

```
       Name  Age      City
p1    Alice   25  New York
p2      Bob   30     Paris
p3  Charlie   35    London
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London
```

Output 8:

```
    Name  Age      City
0  Alice   25  New York
1    Bob   30     Paris
```

Output 9:

```
      Name       Job  Salary
0    Alice   Teacher   25000
1      Bob    Doctor   55000
2  Charlie   Teacher   20000
3    Dylan    Doctor   35000
4     Emma  Engineer   65000
Job
Doctor      45000.0
Engineer    65000.0
Teacher     22500.0
Name: Salary, dtype: float64
```

Program 4:

```
listdata=[
    [1,'Alice',10],
    [2,'Bob',15],
    [3,'Don',25]
]
df = pd.DataFrame(listdata,columns=['ID','Name','Score'])
print(df)
```

Program 5:

```
df = pd.read_csv("medal.csv")
print(df)
```

Program 6:

```
listdata=[
    [1,'Alice',15],
    [2,'Bob',15],
    [3,'Don',45]
]
df = pd.DataFrame(listdata,columns=['ID','Name','Score'])
print("Data frame")
print(df)
# sorting  first by score then by ascendin order of name
sorted_df = df.sort_values(by=['Score','Name'],ascending=[False,True])
print("Sorted Dataframe")
print(sorted_df)
```

Program 7:

```
data = [
    ['Alice', 25, 'New York'],
    ['Bob', 30, 'Paris'],
    ['Charlie', 35, 'London']
]

df = pd.DataFrame(data, columns=['Name', 'Age', 'City'],index=['p1','p2','p3'])
print(df)
df = df.reset_index(drop=True) # drop=True means don’t add the old index as a column.
print(df)
```

Program 8:

```
data = [
    ['Alice', 25, 'New York'],
    ['Bob', 30, 'Paris'],
    ['Charlie', 35, 'London']
]

df = pd.DataFrame(data, columns=['Name', 'Age', 'City'])
print(df.iloc[0:2,:])
```

Output 10:

```
     0  1    2  3
A  1.0  2  NaN  4
B  NaN  2  3.0  4
     0  1    2  3
A  1.0  2  0.0  4
B  0.0  2  3.0  4
```

Output 11:

```
Profit Column Converted to Boolean:
   cname  profit
0     A    True
1     B   False
2     C   False
```

Output 12:

```
    ID   Name  Stipend   Position
0  101   John    30000  Developer
1  102  Amith    35000  Developer
2  103   Sona    20000     Tester
```

Program 9:

```
data = [
    ['Alice','Teacher',25000],
    ['Bob','Doctor',55000],
    ['Charlie','Teacher',20000],
    ['Dylan','Doctor',35000],
    ['Emma','Engineer',65000]
]
df = pd.DataFrame(data, columns=['Name', 'Job', 'Salary'])
print(df)
avg_by_job = df.groupby('Job')['Salary'].mean()
print(avg_by_job)
```

Program 10:

```
data = {
    'A': [1, 2, np.nan, 4],
    'B': [np.nan, 2, 3, 4]
}

df = pd.DataFrame.from_dict(data,orient='index')
print(df)
df = df.fillna(0)
print(df)
```

Program 11:

```
data = {
    "cname": ["A", "B", "C"],
    "profit": [1000, -500, 0]
}
df = pd.DataFrame(data)
df["profit"] = df["profit"] > 0
print("Profit Column Converted to Boolean:\n", df)
```

Program 12:

```
data1 =[
    [101,'John',30000],
    [102,'Amith',35000],
    [103,'Sona',20000]
]
df1 = pd.DataFrame(data1,columns=['ID','Name','Stipend'])
data2 =[
    [101,'Developer'],
    [102,'Developer'],
    [103,'Tester']
]
df2 = pd.DataFrame(data2,columns=['ID','Position'])
df3 = pd.merge(df1,df2,how='inner',on='ID')  #inner join (only matching keys)
print(df3)
```

Result:

The program has been successfully executed and the output is verified.

Output:

```
Accuracy =  0.9473684210526315
```

New Sample belongs to setosa

Exercise 5: KNN

Aim:

Program to implement KNN Algorithm using any standard dataset available in the public domain and find the accuracy of the algorithm.

Algorithm:

1. Choose the number of neighbors (k).

2. Calculate distance between the query point and all training samples.

   Euclidean distance = $\sqrt{\sum_{i=1}^{n}(y_i - x_i)^2}$

3. Sort the distances and select the k nearest neighbors.

4. Aggregate the labels (for classification) or values (for regression):
   - Classification: Use majority voting among the k neighbors.
   - Regression: Take the average of the k neighbors' values.

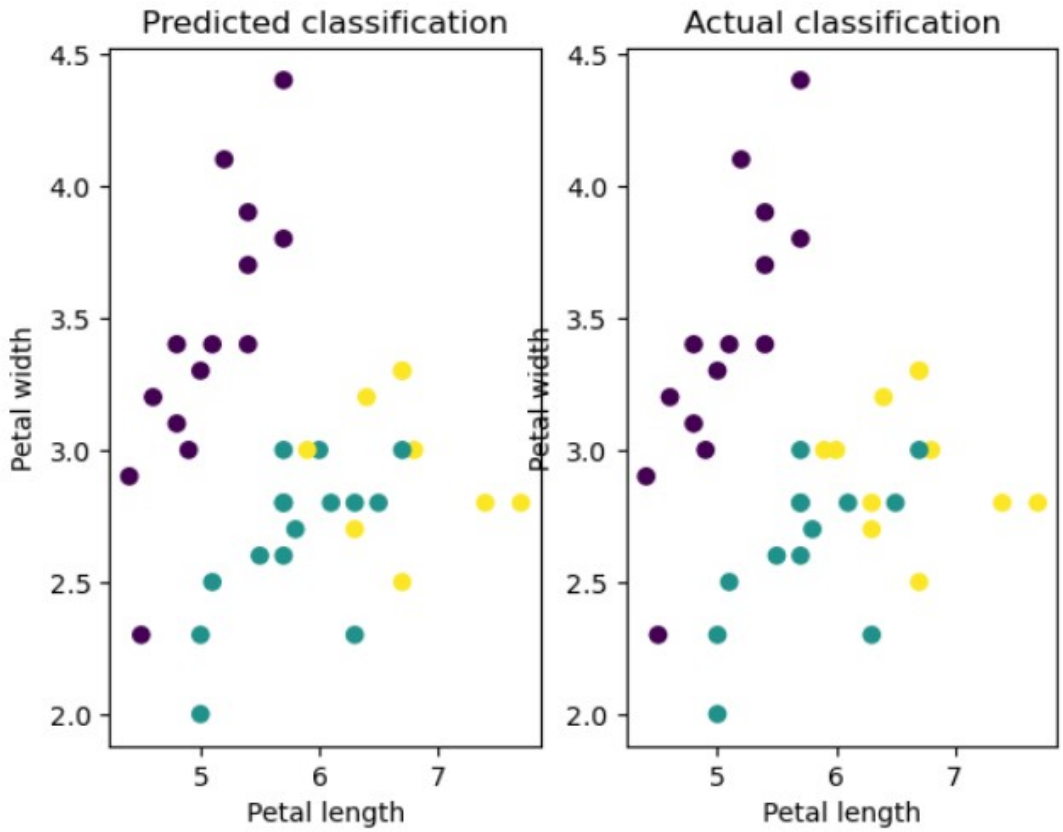
Source Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
import matplotlib.pyplot as plt

iris=load_iris()
x=iris.data
y=iris.target
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.25)
model = KNeighborsClassifier(n_neighbors=5) # setting k=5
model.fit(x_train,y_train)

y_prediction = model.predict(x_test)
print("Accuracy = ",metrics.accuracy_score(y_test,y_prediction)

sample = [[2,2,2,2]] # 1 sample with 4 attribuites - petal length,petal width, sepal length, sepal width
pred = model.predict(sample)[0] # getting first prediction as the index of the flower type
print("New Sample belongs to  ",iris.target_names[pred])
```



plt.subplot(1,2,1)

```
plt.scatter(x_test[:,0],x_test[:,1],c=y_prediction)
plt.title("Predicted classification")
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.subplot(1,2,2)
plt.scatter(x_test[:,0],x_test[:,1],c=y_test)
plt.title("Actual classification")
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.show()
```

Result:

The program has been successfully executed and the output is verified.

Output:

```
1.0

              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        16
  versicolor       1.00      1.00      1.00        14
   virginica       1.00      1.00      1.00        15

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Exercise 6: Implementation of Naive Bayes Classifier

Aim:

Program to implement Naive Bayes Algorithm using any standard dataset available in the public domain and find the accuracy of the algorithm.

Algorithm:

Using Bayes' theorem,

$$P(c_k|X) = \frac{P(X|c_k)P(c_k)}{P(X)}$$

Given A training dataset with n features: (F_1, F_2, ..., F_n). p class labels: {c_1, c_2, ..., c_p}.
A test instance: X = (x_1, x_2, ..., x_n).

1. **Compute Prior Probabilities**: For each class label c_k (where k = 1 to p), calculate: $P(c_k)$ = (Number of training samples with class c_k) / (Total number of training samples).
2. **Compute Conditional Probabilities**: For each feature F_i (i = 1 to n) and each class c_k (k = 1 to p), calculate: $P(f_i \mid c_k)$ = (Number of times feature value f_i appears in class c_k) / (Total number of samples in class c_k) .
3. Compute Class Scores for Test Instance: For the test instance X = (x_1, x_2, ..., x_n), compute the score q_k for each class c_k:
   $\mathbf{q_k = P(x_1 \mid c_k) \times P(x_2 \mid c_k) \times ... \times P(x_n \mid c_k) \times P(c_k)}$
4. **Find Maximum Score**: Identify the class label c_j such that: $\mathbf{q_j = max\{q_1, q_2, ..., q_p\}}$.
5. **Assign Class Label**: Assign the label c_j to the test instance X...

Source Code:

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics
from sklearn.datasets import load_iris
iris=load_iris()
x=iris.data
y=iris.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=40)
gnb=GaussianNB()
gnb.fit(x_train,y_train)
y_pred=gnb.predict(x_test)
acc=metrics.accuracy_score(y_test,y_pred)
report=metrics.classification_report(y_test, y_pred, target_names=iris.target_names)
print(acc)
print(report)
```

Result:

The program has been successfully executed and the output is verified.

Output:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	9
versicolor	0.95	0.95	0.95	19
virginica	0.94	0.94	0.94	17
accuracy			0.96	45
macro avg	0.96	0.96	0.96	45
weighted avg	0.96	0.96	0.96	45

Exercise 7*: *Implementation of Decision Tree Classifier

Aim:

Program to implement Decision tree classifier Algorithm using any standard dataset available in the public domain and find the accuracy of the algorithm.

Algortihm:

1. Create a root node for the decision tree.
2. If all examples in S belong to the same class (e.g., all positive or all negative), return a single-node tree labeled with that class.
3. If the feature set F is empty, return a single-node tree labeled with the most common class in S.
4. For each attribute in F, calculate its Information Gain with respect to S. Select the attribute A with the highest Information Gain.

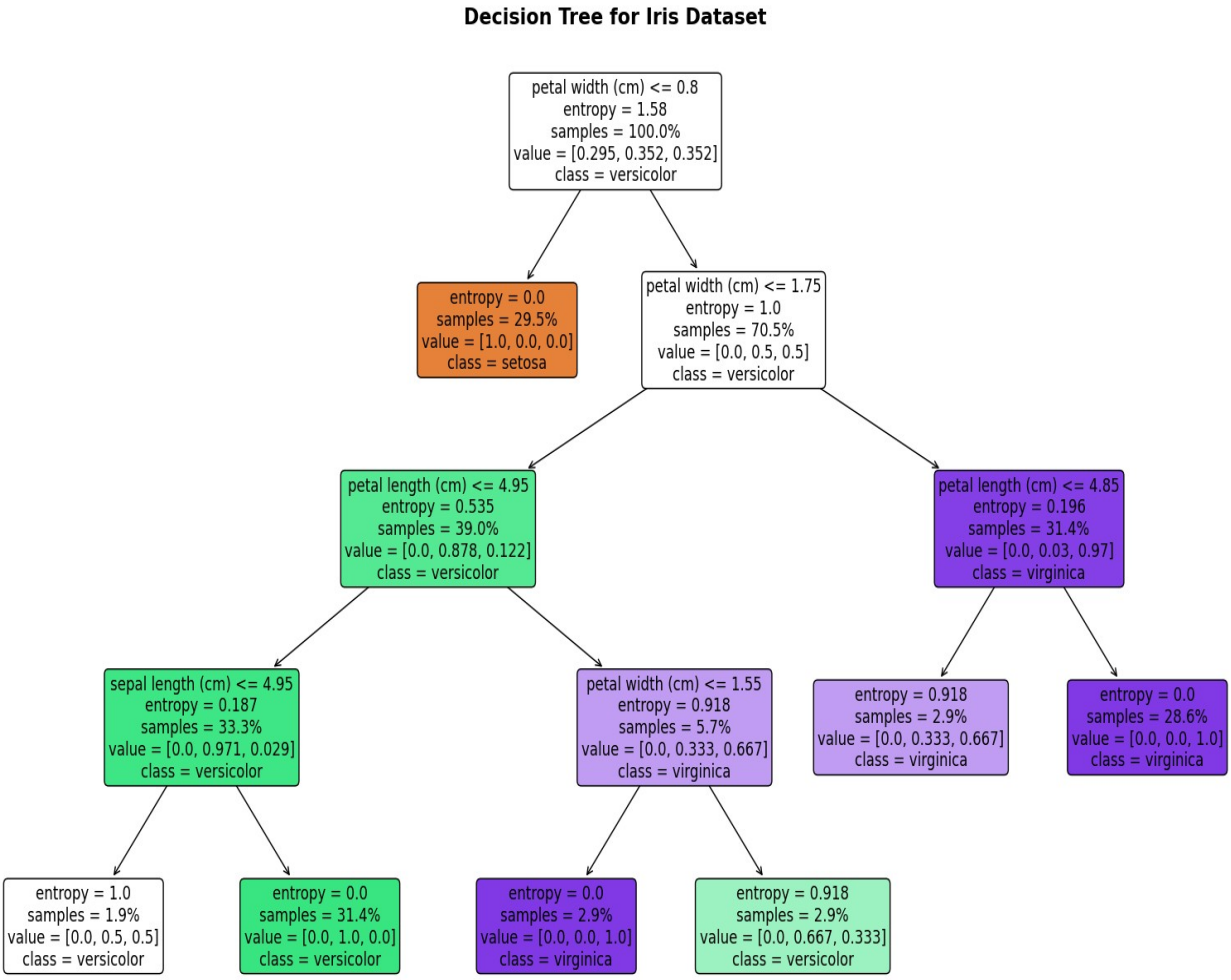
$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \times Entropy(S_v)$$

5. Assign attribute A as the decision attribute for the current node (initially the root).
6. For each possible value v of attribute A, add a branch below the current node labeled "A = v".
7. Let S_v be the subset of examples in S where attribute A equals value v.
8. If S_v is empty, add a leaf node labeled with the most common class in S.
9. If S_v is not empty, recursively call on S_v, C, and F minus attribute A: ID3(S_v, C, F - {A})
10. Return the final decision tree built from these recursive steps.

Source Code:

```
from sklearn.tree import DecisionTreeClassifier
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn import metrics
iris=load_iris()
x=iris.data
y=iris.target
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.3)
clf = DecisionTreeClassifier(criterion="entropy", max_depth=4, min_samples_leaf=2)
clf = clf.fit(x_train, y_train)
y_pred = clf.predict(x_test)
report=metrics.classification_report(y_test, y_pred, target_names=iris.target_names)
print(report)
```


Decision Tree for Iris Dataset
petal width (cm) <= 0.8
entropy = 1.58
samples = 100.0%
value = [0.295, 0.352, 0.352]
class = versicolor
entropy = 0.0
samples = 29.5%
value = [1.0, 0.0, 0.0]
class = setosa
petal width (cm) <= 1.75
entropy = 1.0
samples = 70.5%
value = [0.0, 0.5, 0.5]
class = versicolor
petal length (cm) <= 4.95
entropy = 0.535
samples = 39.0%
value = [0.0, 0.878, 0.122]
class = versicolor
petal length (cm) <= 4.85
entropy = 0.196
samples = 31.4%
value = [0.0, 0.03, 0.97]
class = virginica
sepal length (cm) <= 4.95
entropy = 0.187
samples = 33.3%
value = [0.0, 0.971, 0.029]
class = versicolor
petal width (cm) <= 1.55
entropy = 0.918
samples = 5.7%
value = [0.0, 0.333, 0.667]
class = virginica
entropy = 0.918
samples = 2.9%
value = [0.0, 0.333, 0.667]
class = virginica
entropy = 0.0
samples = 28.6%
value = [0.0, 0.0, 1.0]
class = virginica
entropy = 1.0
samples = 1.9%
value = [0.0, 0.5, 0.5]
class = versicolor
entropy = 0.0
samples = 31.4%
value = [0.0, 1.0, 0.0]
class = versicolor
entropy = 0.0
samples = 2.9%
value = [0.0, 0.0, 1.0]
class = virginica
entropy = 0.918
samples = 2.9%
value = [0.0, 0.667, 0.333]
class = versicolor

```
from sklearn import tree
from sklearn import tree
import matplotlib.pyplot as plt
```

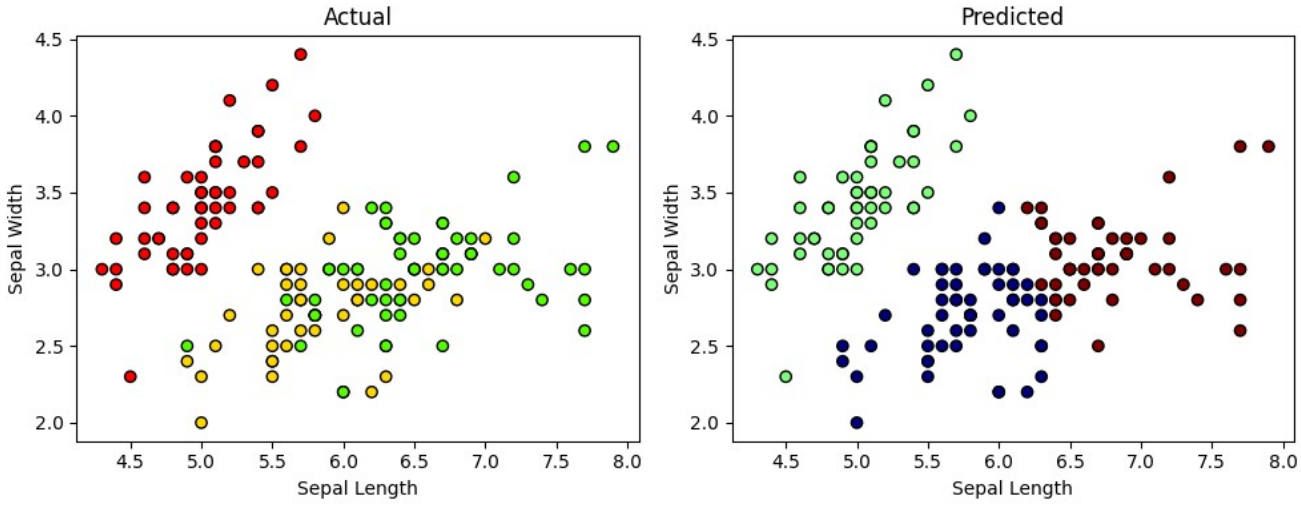
```
fig = plt.figure(figsize=(25,20))
_ = tree.plot_tree(clf, feature_names=iris.feature_names,
 class_names=iris.target_names,filled=True)
```

Result:

The program has been successfully executed and the output is verified.

Output:

```
Adjusted Rand Index (ARI): 0.601
Normalized Mutual Information (NMI): 0.647
```



Exercise 8: K-means Clustering

Aim:

Program to implement K-means Clustering Algorithm using any standard dataset available in the public domain .

Algortihm:

1. Initialize Centers Randomly select k initial cluster centers: v_1, v_2, ..., v_k from the data space.

2. Assign Points to Clusters (Expectation Step) For each data point x_j in X: → Compute the distance (typically Euclidean) between x_j and each cluster center v_i. → Assign x_j to the cluster S_i whose center v_i is closest.

3. Update Centers (Maximization Step) For each cluster i = 1 to k: → Recalculate the center v_i as the mean of all data points assigned to cluster S_i. → Formula: $v_i = (1 / |S_i|) \times$ sum of all x in S_i (where $|S_i|$ is the number of points in cluster S_i)

4. Repeat → Recalculate distances and reassign points based on updated centers. → Continue updating centers and assignments.

5. Check for Convergence → Repeat the assignment and update steps until cluster assignments no longer change (or change very little). → This indicates that the centers have stabilized

Source Code:

```
from sklearn import datasets
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
iris = datasets.load_iris()
X = iris.data[:, :2]
y = iris.target
kmeans = KMeans(n_clusters=3, random_state=21)
kmeans.fit(X)
labels = kmeans.labels_
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='prism', edgecolor='k')
plt.title('Actual')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='jet', edgecolor='k')
plt.title('Predicted')
```

```
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.tight_layout()
plt.show()

from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score
ari = adjusted_rand_score(y, labels)
nmi = normalized_mutual_info_score(y, labels)
print(f"Adjusted Rand Index (ARI): {ari:.3f}")
print(f"Normalized Mutual Information (NMI): {nmi:.3f}")
```

Result:

The program has been successfully executed and the output is verified.

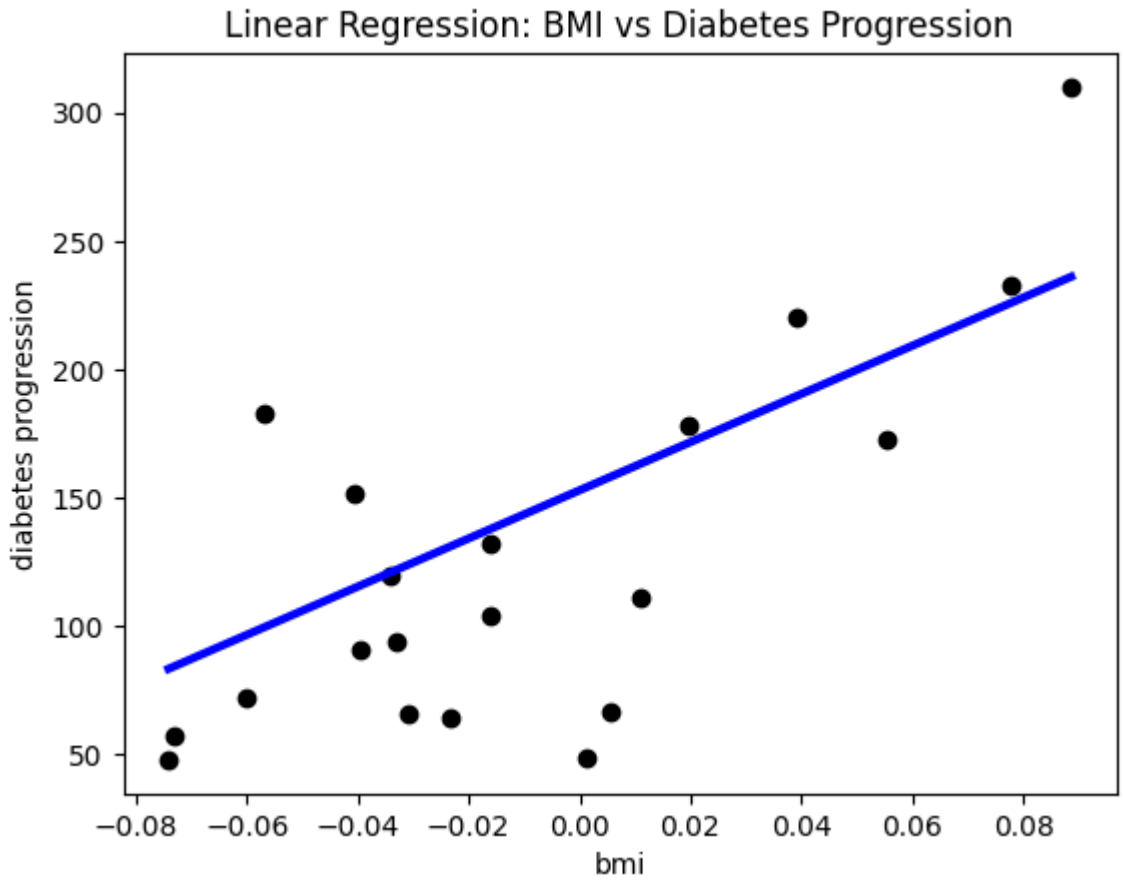
Output:

Coefficients: [938.23786125]

Intercept: 152.91886182616113

Mean squared error: 2548.07

R^2 score: 0.47



Exercise 9: Linear Regression

Aim:

Program to implement Linear Regression Algorithm using any standard dataset available in the public domain.

Algortihm:

1. Load and Prepare Data
2. Input the dataset containing independent variable (X) and dependent variable (Y).
3. Calculate the Means

$$\bar{X} = \frac{\sum X}{n}, \quad \bar{Y} = \frac{\sum Y}{n}$$

4. Compute the slope (b_1) and intercept (b_0) using the formulas

$$b_1 = \frac{\sum (X - \bar{X})(Y - \bar{Y})}{\sum (X - \bar{X})^2}$$

$$b_0 = \bar{Y} - b_1 \bar{X}$$

5. Form the regression equation

$$\hat{Y} = b_0 + b_1 X$$

6. Use the regression equation to predict Y values for given X.

Source Code:

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score
diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)
diabetes_X = diabetes_X[:, np.newaxis, 2]
diabetes_X_train = diabetes_X[:-20]
diabetes_X_test = diabetes_X[-20:]
diabetes_y_train = diabetes_y[:-20]
diabetes_y_test = diabetes_y[-20:]
regr = linear_model.LinearRegression()
regr.fit(diabetes_X_train, diabetes_y_train)
diabetes_y_pred = regr.predict(diabetes_X_test)
print("Coefficients:", regr.coef_)
print("Intercept:", regr.intercept_)
print("Mean squared error: %.2f" % mean_squared_error(diabetes_y_test, diabetes_y_pred))
print("R^2 score: %.2f" % r2_score(diabetes_y_test, diabetes_y_pred))
plt.scatter(diabetes_X_test, diabetes_y_test, color="black")
```

```
sorted_idx = np.argsort(diabetes_X_test[:, 0])
```

```
plt.plot(diabetes_X_test[sorted_idx], diabetes_y_pred[sorted_idx], color="blue", linewidth=3)
plt.xlabel("BMI")
plt.ylabel("Diabetes progression")
plt.title("Linear Regression: BMI vs Diabetes Progression")
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.show()
```

Result:

The program has been successfully executed and the output is verified.

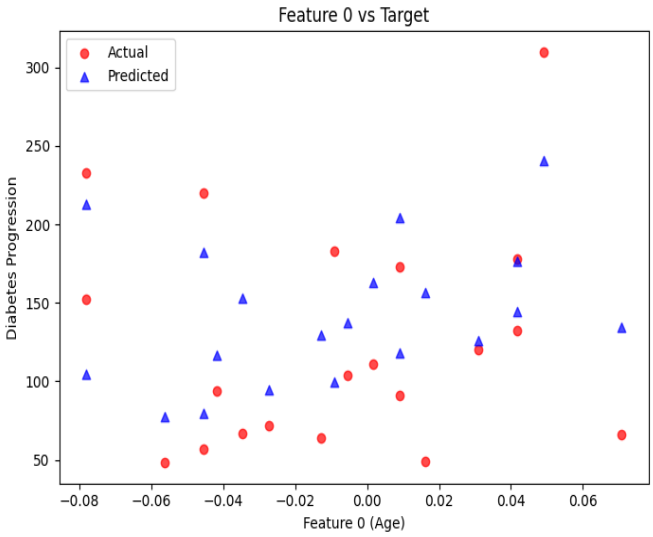
Output:

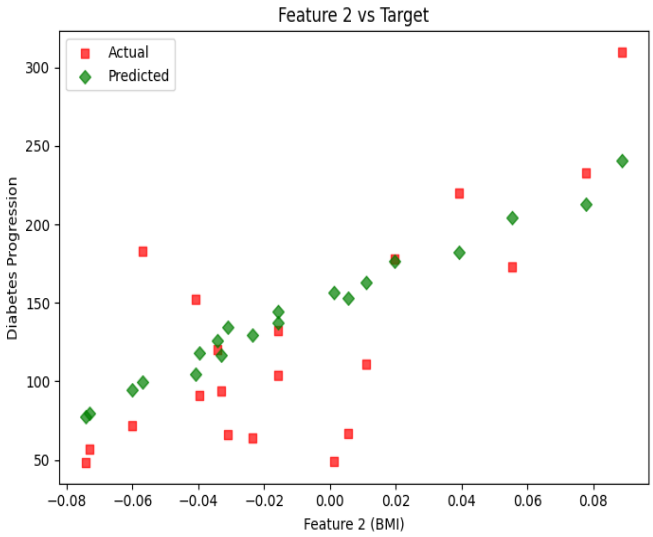
Coefficients: [139.20420118 912.45355549]

Intercept: 152.87670001405584

Mean squared error: 2596.60

R^2 score: 0.46





Exercise 10*: *Multiple Regression

Aim:

Program to implement Multiple Regression Algorithm using any standard dataset available in the public domain.

Algortihm:

1. Load and Prepare Data
2. Input the dataset with independent variables X1,X2,..., Xn and dependent variable Y.
3. Represent the model:

$$Y = b_0 + b_1X_1 + b_2X_2 + ... + b_nX_n$$

4. Arrange data in matrix form

$$Y = XB + E$$

5. Compute coefficients using Normal Equation:

$$B = (X^T X)^{-1} X^T Y$$

6. Form the regression equation:

$$\hat{Y} = b_0 + b_1X_1 + b_2X_2 + ... + b_nX_n$$

7. Evaluate model performance using MSE or R^2.

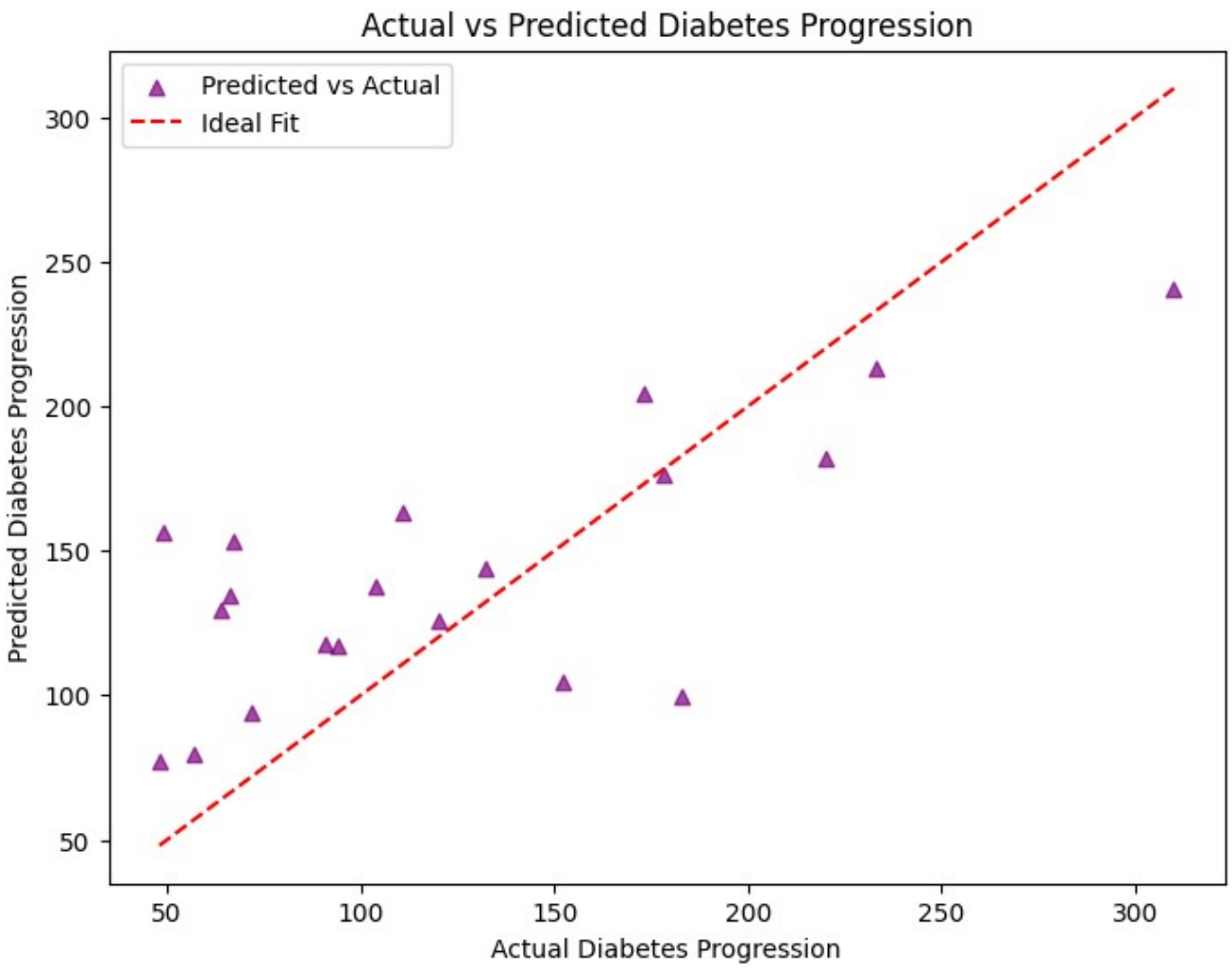
Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score

diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)
diabetes_X = diabetes_X[:, [0, 2]]

diabetes_X_train = diabetes_X[:-20]
diabetes_X_test = diabetes_X[-20:]
diabetes_y_train = diabetes_y[:-20]
diabetes_y_test = diabetes_y[-20:]

regr = linear_model.LinearRegression()
regr.fit(diabetes_X_train, diabetes_y_train)
diabetes_y_pred = regr.predict(diabetes_X_test)
```


Actual vs Predicted Diabetes Progression
Predicted vs Actual
Ideal Fit
Predicted Diabetes Progression
Actual Diabetes Progression
50
100
150
200
250
300

```
print("Coefficients:", regr.coef_)
print("Intercept:", regr.intercept_)
print("Mean squared error: %.2f" % mean_squared_error(diabetes_y_test, diabetes_y_pred))
print("R^2 score: %.2f" % r2_score(diabetes_y_test, diabetes_y_pred))

plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
plt.scatter(diabetes_X_test[:, 0], diabetes_y_test, color='red', marker='o', label='Actual', alpha=0.7)
plt.scatter(diabetes_X_test[:, 0], diabetes_y_pred, color='blue', marker='^', label='Predicted',
alpha=0.7)
plt.xlabel('Feature 0 (Age)')
plt.ylabel('Diabetes Progression')
plt.title('Feature 0 vs Target')
plt.legend()

plt.subplot(1, 2, 2)
plt.scatter(diabetes_X_test[:, 1], diabetes_y_test, color='red', marker='s', label='Actual', alpha=0.7)
plt.scatter(diabetes_X_test[:, 1], diabetes_y_pred, color='green', marker='D', label='Predicted',
alpha=0.7)
plt.xlabel('Feature 2 (BMI)')
plt.ylabel('Diabetes Progression')
plt.title('Feature 2 vs Target')
plt.legend()

plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 6))
plt.scatter(diabetes_y_test, diabetes_y_pred, color='purple', marker='^', label='Predicted vs Actual',
alpha=0.7)
plt.plot([diabetes_y_test.min(), diabetes_y_test.max()],
        [diabetes_y_test.min(), diabetes_y_test.max()], 'r--', label='Ideal Fit')
plt.xlabel("Actual Diabetes Progression")
plt.ylabel("Predicted Diabetes Progression")
plt.title("Actual vs Predicted Diabetes Progression")
plt.legend()
plt.show()
```

Result:

The program has been successfully executed and the output is verified.